

THE PADDLER

Project Report and Technical Handover

Project	Custom streetwear ecommerce platform
Updated	23 May 2026
Stack	Next.js, TypeScript, Firebase, Razorpay, Shiprocket
Status	Main website complete; domain/live production setup pending

This document summarizes the current build, integrations, completed modules, operating logic, and launch checklist for THE PADDLER.

Executive Summary

THE PADDLER is a custom ecommerce website for a streetwear brand. The platform is built as a full-stack Next.js application with Firebase for authentication/data, Razorpay for payments, Shiprocket for logistics order creation, and an admin panel for store operations.

The main website flow is now functionally complete. Customers can browse, sign in, manage addresses, apply coupons, pay through Razorpay, see a thank-you page, download invoices, track order status, request cancellation before shipment, and request returns after delivery. Admin and staff workflows cover product management, inventory, orders, returns, banners, reports, maintenance mode, and shipment order creation.

Technology Foundation

- Frontend and backend framework: Next.js 16 App Router
- Language: TypeScript
- UI: React 19, Tailwind CSS, custom animated interfaces
- Database: Firebase Cloud Firestore
- Authentication: Firebase Authentication
- Payments: Razorpay
- Logistics: Shiprocket
- Email: SMTP/Nodemailer
- Hosting target: Vercel
- Domain target: thepaddler.in

Customer Website

Completed customer-facing modules:

- Animated landing page with hero slider, featured products, brand storytelling sections, countdown, and WhatsApp contact
- Mobile-aware navigation with menu, logo, cart, profile, and wishlist behavior
- Shop page with product cards, tags, MRP strike-through, selling price, wishlist, and responsive grid
- Product detail page with image gallery, image zoom, size selection, color display, coupon check, quantity controls, delivery estimate, add-to-cart, and buy-now actions

- Cart and checkout pages with saved address selection, coupon auto-apply, shipping calculation, and Razorpay payment
- Thank-you page shown after successful payment
- Customer orders page with invoice download, payment status, order status animation, cancellation rules, retry payment, and return request flow
- Legal pages for privacy policy, terms and conditions, shipping policy, returns, and refund expectations

Authentication and User Data

Completed authentication work:

- Google login
- Email/password signup and login
- Re-enter password confirmation
- Password visibility toggle
- Phone number capture in +91 XXXXX XXXXX format
- Firebase phone OTP verification
- Firestore user profile creation on signup/login
- Admin user page separates registered-only users from customers who have placed orders
- Role-based access rules for admin, staff, and customer

Current role model:

- Admin: full access to admin panel and site controls
- Staff: operational access for products, inventory, banners, featured products, orders, returns, and selected editing tasks
- Customer: no admin access

The influencer role/dashboard was removed from the active admin model.

Products and Inventory

Completed product work:

- Product creation and editing from admin
- Image upload fields instead of plain image-path entry

- Multiple product images support
- Product badges such as new arrival, bestseller, and limited
- Featured product selection from admin
- Product price, MRP, discount preview, and storefront price display
- Shared inventory model by color and size
- Inventory deduction after successful payment only

Important inventory logic:

When several designs use the same base color and size stock, selling one item deducts the shared stock for that color-size group. For example, if four black designs share black M stock, a sale of one black M item reduces the available black M inventory across those designs.

Coupons

Completed coupon work:

- Coupon creation and management in admin
- Product page coupon check
- Coupon persists into checkout after customer checks it on product page
- Auto-apply behavior on checkout
- Coupon usage reflected in order data
- Coupon can be used once per user for lifetime
- Coupon revenue/usage appears in admin where relevant

Payments

Current active gateway: Razorpay.

Completed payment flow:

1. Customer places an order from checkout.
2. The order is created in Firestore as pending until payment succeeds.
3. Razorpay payment opens and processes the payment.
4. Backend verification confirms the payment.
5. Order payment status becomes success.

6. Inventory is deducted.
7. Invoice data is attached.
8. Customer is redirected to thank-you page.
9. Admin and customer order pages show updated status.
10. Shiprocket order creation is triggered after successful payment.

PhonePe is no longer part of the active checkout flow.

Shiprocket Logistics

Completed Shiprocket work:

- Backend-only Shiprocket API integration
- Credentials stored in <code>.env.local</code> locally and Vercel environment variables in production
- Shiprocket authentication handling
- Shiprocket order creation after successful Razorpay payment
- Shipment/order IDs saved in Firestore
- Live delivery/pincode service can use Shiprocket when credentials are available
- Courier partner selection remains manual inside Shiprocket

Important logistics rule:

The website creates the Shiprocket order, but it does not automatically mark the customer order as shipped. The order remains in the correct internal status chain until admin updates it.

Status chain:

1. Paid
2. Processing
3. Shipped
4. Transit
5. Delivered

Direct jumps are blocked. An order cannot move to transit before shipped, and cannot move to delivered before transit.

Orders, Invoices, Cancellations, and Returns

Completed order features:

- Unique invoice number per order
- Invoice download
- Admin order view with filters
- Manual admin status movement through the allowed chain
- Customer cancellation available before shipment
- Admin cancellation available until shipment
- Cancel button disappears after shipment
- Return button appears after delivery
- Return is available for 3 days after delivery
- Return form captures reason, description, and image
- Return requests are visible in admin
- Unread return request count appears on admin tab

Return policy implemented:

Returns and replacements are accepted within 3 days of delivery, only for size-related cases and only when the product is unused, unwashed, and in original packaging.

Admin Panel

Completed admin modules:

- Dashboard
- Products
- Inventory
- Orders
- Users
- Coupons
- Returns
- Banner control
- Featured products

- Homepage content editor
- Countdown editor
- Maintenance mode
- Download report

Admin report flow:

- Admin selects a date range or all-time report.
- System generates a PDF report.
- Report includes order, customer, invoice, payment, address, and transaction details in tabular format.
- Reset functionality was removed and replaced with report download.

UI and Experience Work

Completed visual improvements:

- Larger brand logo in navbar
- Mobile navigation cleanup
- Animated landing page hero and scrolling effects
- Featured product card polish
- Product image zoom
- Animated order timeline
- Thank-you page animation
- Homepage sections upgraded from plain static blocks to stronger brand presentation
- Responsive fixes for mobile overflow and section clipping

Environment and Deployment

Local environment:

- `.env.local` contains local-only secrets and must not be committed.
- Razorpay and Shiprocket variables are required for payment and logistics.
- Firebase public config remains in the client setup.

Production environment:

- | |
|--|
| - Vercel environment variables must be updated whenever keys change. |
| - After updating Vercel environment variables, redeploy the project. |
| - Firebase authorized domains must include final production domains. |

Remaining Work Before Final Launch

Primary remaining items:

- | |
|--|
| - Connect final domain thepaddler.in and www.thepaddler.in |
| - Add final domain to Firebase authorized domains |
| - Switch Razorpay from test keys to live keys after approval |
| - Verify Razorpay payment with live mode |
| - Confirm Shiprocket production credentials and pickup address |
| - Run one complete live test order |
| - Confirm invoice, email, Firestore, inventory, Razorpay, Shiprocket, and admin/customer order views |
| - Final mobile QA on product, cart, checkout, login, and orders pages |
| - Confirm legal pages match final business policy |

Optional future improvements:

- | |
|--|
| - Shiprocket webhook tracking sync |
| - Automatic AWB assignment using a selected/default courier |
| - Bulk product import |
| - Bulk pincode upload from a prepared sheet |
| - Advanced analytics for conversion, product views, and repeat customers |
| - Customer email/SMS notification templates |

Final Assessment

Except for domain/live-key setup and final production testing, the main website build is complete. The platform now has customer shopping, payments, admin operations, inventory control, coupon enforcement, returns, reports, and Shiprocket order creation in place.